

Summary: Dynamic Memory Allocation in C — Functions, Implementation, and Resource Management

This video presents a structured analysis of Dynamic Memory Allocation (DMA) in C, focusing on runtime memory management using the heap. It consolidates the behavior, implementation logic, and practical implications of the four primary DMA functions: `malloc`, `calloc`, `realloc`, and `free`. Emphasis is placed on memory initialization, resizing strategies, performance trade-offs, and the prevention of memory leakage. Correct usage of these functions is critical for developing efficient, reliable software that interacts safely with the operating system.

1. Primary Allocation Functions: `malloc` and `calloc`

Dynamic memory allocation enables programs to request memory during execution rather than relying on fixed-size static arrays. The two fundamental allocation functions are `malloc` and `calloc`.

1.1 `malloc` Implementation

The `malloc` function allocates a single contiguous block of memory of a specified size (in bytes) from the heap.

Key Characteristics

Syntax Logic

Memory size is typically calculated as:

`n * sizeof(data_type)`

-
- **Return Value**
 - Returns a pointer to the starting address of the allocated block.
 - Returns **NULL** if sufficient heap memory is unavailable.
- **Validation Requirement**

The returned pointer should always be checked against **NULL** before use to avoid runtime errors.

1.2 Comparison: `calloc` vs. `malloc`

While both functions allocate memory dynamically, their behavior differs in initialization and performance.

Feature	<code>malloc</code>	<code>calloc</code>
Arguments	Single argument (total bytes)	Two arguments (number of blocks, size per block)
Initialization	Memory not initialized (contains garbage values)	All bits initialized to zero
Performance	Faster (less overhead)	Slower / more expensive
Reliability	Lower (unknown initial state)	Higher (predictable initial state)

Key Insight

The selection between `malloc` and `calloc` represents a trade-off between execution speed and data reliability.

`calloc` is characterized as “expensive but reliable.”

2. Resizing Memory with `realloc`

The `realloc` function enables resizing of a previously allocated memory block, supporting both expansion and reduction.

2.1 Usage Prerequisites

- `realloc` must be applied only to memory that was previously allocated using `malloc` or `calloc`.
- It cannot be used independently.

2.2 Expansion Scenarios (Increasing Size)

When requesting a larger memory block, the system follows one of two logic paths:

Case 1: Contiguous Memory Available

- If sufficient free space exists immediately after the current block:
 - The memory block is extended.
 - The original address is returned.

Case 2: Contiguous Memory Unavailable

- The system searches the heap for a new block of the requested size.
- **Automatic Data Copying**
 - Existing data is copied from the old location to the new location.
 - The old block is then deallocated.
- **Return Value**
 - A pointer to the new memory location is returned.

2.3 Shrinking Memory Size

- The allocated block size is reduced.
- The truncated portion is no longer accessible.
- This behavior is better interpreted as an intentional reduction of memory footprint, not accidental data loss.

3. Memory Leakage and the **free** Function

In C, memory management is entirely manual. Improper handling of heap memory leads to memory leakage and system instability.

3.1 Mechanism of Memory Leakage

Memory leakage occurs when allocated heap memory becomes unreachable but remains occupied.

Typical Scenario

1. A function dynamically allocates memory and stores the address in a local pointer.
2. The function terminates execution.
3. The local pointer is destroyed (removed from the stack).
4. The heap memory address is lost permanently.

Consequences

- The program cannot access or release the allocated memory.
- Repeated leaks gradually exhaust heap space.
- The operating system eventually reports “Memory Not Available”, even if the program is no longer using that memory.

3.2 The **free** Function

The **free** function prevents memory leakage by:

- Deallocating the memory block pointed to by its argument.
- Returning the memory to the operating system or making it available for future allocations within the program.

Proper usage of **free** is essential for maintaining long-running, stable applications.